

Movie Theater Ticketing System (MTTS)

Data Management Strategy

Course Project Deliverable 4 — Section 6 of the MTTS SRS

Benjamin Nguyen

Sole Author and Developer

Gus Hanna, Ph.D.

Instructor, CS 250

San Diego State University

August 4, 2026

6. Data Management Strategy

This section is the fourth deliverable of the course project. It extends the Movie Theater Ticketing System (MTTS) Software Requirements Specification with an updated software architecture diagram that shows the system's databases, and with a full description of how MTTS stores, organizes, protects, and recovers its persistent data.

Sections 1 through 3 defined what the system must do, Section 4 defined the software design as a set of classes and their operations, and Section 5 defined how that design is tested. This section answers the remaining question those sections deliberately deferred: where the data actually lives. Every design decision below is stated together with the alternative that was considered and the tradeoff that was accepted, because the choice of a data store is difficult and expensive to reverse once the system is in production.

6.1 Overview and Design Goals

MTTS is a browser-based ticketing system for a San Diego–area chain of twenty theaters. It sells up to twenty tickets per transaction across roughly 6 to 8 showtimes per screen per day, supports assigned seating in deluxe auditoriums, and must serve at least 1,000 concurrent users while showing the website and the in-theater kiosks the same live availability data. Those requirements drive five data management goals:

- Correctness under concurrency. Two customers must never be sold the same assigned seat, and the total tickets sold for a showtime must never exceed auditorium capacity, even when hundreds of users check out at the same moment.
- A single source of truth. The kiosk and the website are the same application talking to the same data, so availability cannot diverge between channels.
- Low-latency reads. Browsing movies, showtimes, and seat maps is by far the most frequent operation and must stay responsive at peak load.
- Protection of sensitive data. Customer accounts, contact information, and payment references are personally identifiable or financially sensitive and must be encrypted, minimized, and auditable.
- Recoverability. A hardware or software failure must not destroy sold-ticket records, because a lost ticket record means a customer is turned away at the door.

6.2 Updated Software Architecture Diagram

Figure 6.1 updates the architecture first presented in Section 4.2 by expanding the previously undifferentiated "central database" into a concrete data tier. The three-tier structure is unchanged: thin clients hold no business logic, a single C++ application server holds all of it, and the data tier is reached only through the server's Data Access Layer.

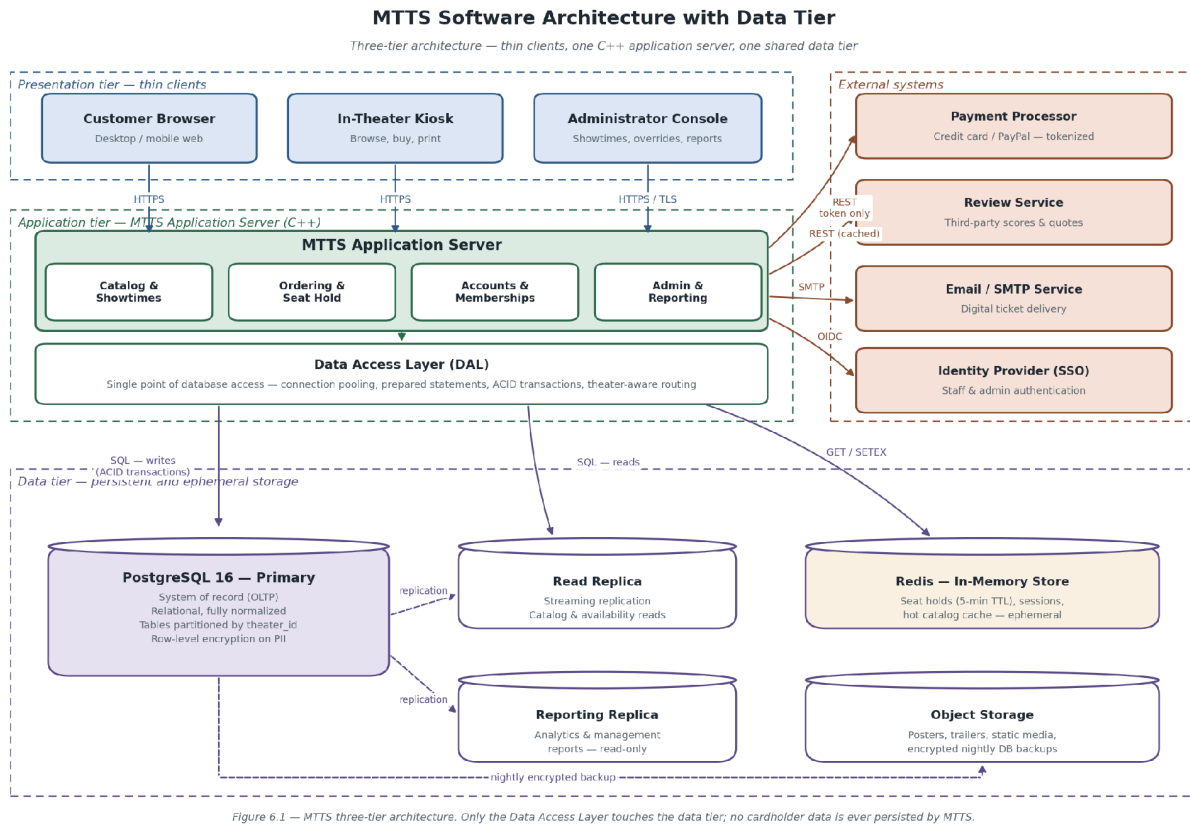


Figure 6.1 — MTTs three-tier architecture, with the data tier expanded to show the primary database, its replicas, the in-memory store, and object storage.

6.2.1 Component Responsibilities in the Data Tier

Component	Responsibility	Data it holds
Data Access Layer (DAL)	The only component permitted to open a database connection. Owns connection pooling, prepared statements, transaction boundaries, and theater-aware routing.	None — stateless
PostgreSQL primary	System of record for every durable fact in the system. Accepts all writes and any read that must be strictly current.	Catalog, sales, customer, and audit data
Read replica	Serves high-volume browsing reads (movie lists, showtime grids, poster metadata) through streaming replication from the primary.	Read-only copy of the primary
Reporting replica	Serves the management reports required by Section 3.2 (tickets sold and ticket price per movie) so that analytical queries never compete with checkout traffic.	Read-only copy of the primary

Component	Responsibility	Data it holds
Redis in-memory store	Holds only ephemeral state: the five-minute seat holds, web and kiosk session tokens, and a short-lived cache of third-party review scores.	Seat holds, sessions, cache
Object storage	Stores large binary assets and encrypted nightly database backups, which do not belong in relational rows.	Posters, trailers, backups

6.2.2 Changes from the Section 4 Architecture

Three changes were made to the earlier diagram, each of them motivated by a data management concern rather than by a change in functionality:

- The single database box became a primary with two read replicas. Section 4 showed one database, which would have forced 1,000 concurrent browsers, the kiosks, and management reporting onto the same instance that processes checkouts.
- An in-memory store was added. Section 4 modeled the seat hold as a `SeatHold` class without saying where a hold lives; holding a seat for five minutes is short-lived, high-churn state, and writing every hold and expiry to durable storage would generate write traffic that has no long-term value.
- Object storage was separated from the database. Posters and trailers are large, immutable, and served directly to browsers, so keeping them out of the relational store keeps backup sizes and query working sets small.

6.3 Data Management Strategy

6.3.1 Technology Selection: A Relational (SQL) Database

MTTS uses a relational SQL database — PostgreSQL 16 — as its system of record. The data this system manages is highly structured and highly interrelated: a ticket is meaningless without its showtime, a showtime without its auditorium, an auditorium without its theater. That shape is exactly what the relational model was designed for, and five specific requirements make it the right choice here.

- Transactional integrity. Selling a ticket writes an order, one or more ticket rows, a payment record, and a decrement of seat availability. These writes must all succeed or all fail. ACID transactions give this guarantee in the database itself, rather than requiring the application to implement compensation logic for partial failures.
- Enforceable constraints. A unique constraint on `(showtime_id, seat_id)` makes double-booking an assigned seat impossible at the storage layer, and a unique constraint on the ticket barcode enforces the SRS requirement that every ticket be uniquely identifiable. A constraint enforced by the database cannot be bypassed by an application bug.

- Ad hoc reporting. Management reporting requirements are stated in general terms and will grow. SQL answers questions such as tickets sold per movie per theater per week without any schema change or new application code.
- Predictable joins across subject areas. Nearly every screen in the system joins three or more entities. Doing those joins in the database is far cheaper than assembling them in the application server.
- Operational maturity. PostgreSQL is free, is well understood, has first-class C++ client libraries (libpqxx) that suit the server implementation chosen in Section 4, and provides built-in streaming replication and point-in-time recovery, all of which matter for a four-month, \$500,000 project that cannot afford to build infrastructure from scratch.

6.3.2 Number of Databases

MTTS uses one logical database plus one specialized in-memory store.

The single relational database holds all durable data for all twenty theaters. It is deployed physically as one primary instance and two read replicas, but these replicas are copies of the same logical database, not independent databases: they hold no data of their own and accept no writes. Alongside it, Redis holds ephemeral state only. Redis was chosen for this role deliberately: seat holds have a natural five-minute time-to-live, and an expiring key is a far simpler and more reliable expiry mechanism than a background job sweeping a table of abandoned holds.

The important consequence of this decision is that nothing durable is stored in two places. If the Redis instance is lost, every hold is released and every session is ended — customers must sign in again and re-select seats — but no sold ticket, no order, and no account is affected. This asymmetry is the reason a second store is acceptable here: it holds only data the system is willing to lose.

6.3.3 Logical Organization of the Data

Within the single database, the schema is organized into four subject areas. Figure 6.2 shows the entities, their keys, and the relationships between them.

MTTS Logical Data Model

Relational schema of the single MTTs database (colour = logical subject area); dashed entity is ephemeral state held in Redis

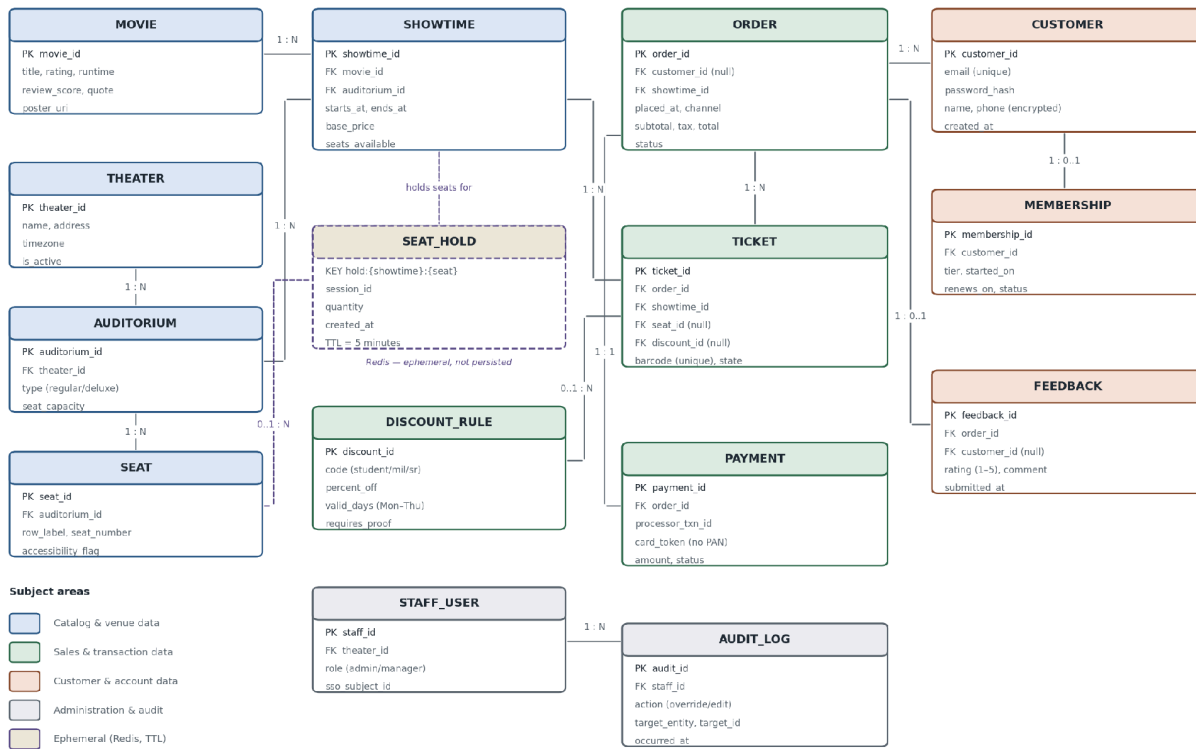


Figure 6.2 — Logical data model. One relational database holds all four subject areas; only SEAT_HOLD lives outside it, in Redis, and expires automatically.

Figure 6.2 — MTTs logical data model. Colour indicates subject area; the dashed entity is ephemeral state held in Redis rather than in the relational database.

Subject area	Entities	Why it is grouped this way
Catalog and venue	MOVIE, THEATER, AUDITORIUM, SEAT, SHOWTIME	Read-heavy, slowly changing reference data. It is written only by administrators and read on nearly every page, which makes it the natural candidate for replica reads and caching.
Sales and transactions	ORDER, TICKET, PAYMENT, DISCOUNT_RULE	Write-heavy, append-mostly, and financially significant. These are the tables that must be transactionally consistent and are never served from a replica.
Customer and account	CUSTOMER, MEMBERSHIP, FEEDBACK	The only tables containing personally identifiable information, which lets encryption, access restriction, and retention rules be applied to a small, clearly bounded set of columns.
Administration and audit	STAFF_USER, AUDIT_LOG	Separates privileged identities and the immutable record of administrator overrides from ordinary customer data.

Grouping the schema this way is a logical organization, not a physical split. The tables live in one database and can be joined freely, but each group has a distinct read/write profile, a distinct sensitivity level, and a distinct retention rule, which is precisely what makes the grouping useful when access control and backup policy are applied.

6.3.4 Partitioning and Scale

The two tables that grow without bound are TICKET and ORDER: twenty theaters, roughly eight showtimes per screen per day, and up to 225 seats per screening produce on the order of a few million ticket rows per year. Both tables are declaratively partitioned by theater_id, and SHOWTIME is partitioned the same way.

Partitioning by theater matches how the data is actually queried, because a customer browsing or buying is always working within one theater and staff are almost always working within their own location. The database therefore scans one partition rather than the whole table, and an index rebuild or an archival of last year's tickets can be done one theater at a time without locking the others. Partitioning is transparent to the application: the Data Access Layer issues ordinary SQL and includes theater_id in the predicate, which is what the Section 4 design already described as theater-aware routing.

6.3.5 Concurrency Control and the No-Oversell Guarantee

The hardest correctness requirement in the system is that a seat is never sold twice. MTTS enforces it in three layers, from cheapest to strongest:

- Redis hold. When a customer selects seats, the server writes a hold key for each seat with a five-minute TTL using a set-if-not-exists operation. If the key already exists, another customer is mid-checkout on that seat and it is shown as unavailable. This resolves the vast majority of contention without touching the database.
- Database transaction. At checkout, the order, tickets, and payment record are written inside one transaction that selects the affected showtime row for update, so concurrent checkouts on the same showtime serialize rather than interleave.
- Unique constraint. The pair (showtime_id, seat_id) is unique in the TICKET table. Even if the hold expired at the wrong instant and two transactions raced, the second insert fails and its transaction rolls back. The customer sees a clear "seat no longer available" message rather than a double sale.

The third layer is what makes the guarantee unconditional. The first two layers exist so that it almost never has to fire.

6.3.6 Sensitive Data

The single most important decision about sensitive data in MTTS is what the system refuses to store. Card numbers are never persisted and never pass through the application server's own storage: the browser or kiosk submits card details directly to the external payment processor, which returns a token, and only that token and the processor's transaction identifier are written to the PAYMENT table. This keeps the system out of the scope of most cardholder-data handling obligations and means a database compromise cannot expose a single card number.

Data	Classification	Control applied
Card number, CVV	Prohibited	Never stored, never logged. Tokenized by the external processor; only the token is retained.
Customer name, email, phone	Personally identifiable	Encrypted at rest at the column level; returned only to the account owner or to an authenticated staff user acting on that order.
Account password	Secret	Never stored. Only a salted hash (Argon2id) is retained; the plaintext is not recoverable by anyone, including administrators.
Ticket barcode	Sensitive	Randomly generated, not sequential, so a valid barcode cannot be guessed from another one.
Session and hold keys	Ephemeral	Held only in Redis with a short TTL; never written to durable storage.
Administrator overrides	Auditable	Every override or showtime edit writes an immutable AUDIT_LOG row identifying the staff member, the action, the target, and the timestamp.

Three further controls apply across the whole data tier. All traffic between tiers uses TLS, so data is encrypted in transit as well as at rest. The database accounts used by the application follow least privilege: the reporting account can read only the reporting replica and cannot see PII columns at all. Finally, feedback and order records older than seven years are archived and PII columns are erased, so the system does not retain identifiable data longer than the business needs it.

6.3.7 Availability, Backup, and Recovery

The primary streams its write-ahead log continuously to both replicas, so a replica can be promoted if the primary fails; the recovery time objective is fifteen minutes. A full encrypted backup is written to object storage nightly and write-ahead log segments are archived continuously, giving point-in-time recovery with a recovery point objective of five minutes. Backups are restored into a staging environment monthly, because a backup that has never been restored is only a hypothesis.

Redis is deliberately excluded from the backup plan. Its contents are recoverable by definition — holds simply expire and sessions are re-established — and treating it as disposable is what keeps the recovery procedure simple.

6.4 Alternatives Considered and Tradeoffs

Two independent decisions were made in Section 6.3: which storage technology to use, and how to organize the data across databases. Each had credible alternatives.

6.4.1 Technology Alternatives

Alternative	What it would have offered	Why it was not chosen
Document store (MongoDB)	A showtime and its seat map could be stored as one document, making the common read a single lookup with no joins, and horizontal scaling is simpler.	Multi-document transactions are available but are not the natural mode of use, and the no-oversell guarantee depends on exactly that. The schema is also strongly relational, so document modeling would duplicate movie and theater data and create update anomalies.
Key-value store only (Redis or DynamoDB)	The lowest possible read latency and effortless horizontal scale for the browsing path.	No ad hoc querying, so every management report would require new application code and a purpose-built index. Financial records also demand durability guarantees stronger than an in-memory store provides.
Embedded database (SQLite)	Zero administration and no separate server process, which is attractive for a small project.	Single-writer locking makes it unable to sustain 1,000 concurrent users, and it cannot be replicated for availability. It remains the right choice for the developer test fixtures described in Section 5, and is used there.
Distributed SQL (CockroachDB, Spanner)	ACID transactions with horizontal scale and multi-region availability already built in.	The workload — one metropolitan chain, twenty theaters — fits comfortably on a single primary. The added operational complexity and licensing or hosting cost are not justified within the project's \$500,000 budget.

6.4.2 Data Organization Alternatives

Alternative	What it would have offered	Why it was not chosen
One database per theater (twenty databases)	Perfect isolation between locations, and one theater's load or outage could not affect another.	Chain-wide queries — a customer searching for a movie across the city, or a report of tickets sold company-wide — would require querying twenty databases and merging results in the application. Table partitioning delivers most of the isolation benefit without that cost, which is why it was chosen instead.
Database per service (microservices)	Independent deployment and scaling of catalog, ordering, and account services.	It would move the ticket-purchase transaction across service boundaries, replacing a single ACID transaction with a distributed one. For a solo developer on a four-month schedule, that is a large increase in complexity for a scale the system does not yet need.
Separate operational and analytical databases	Reporting queries could never affect checkout performance at all.	This is effectively what the reporting replica achieves, at a fraction of the cost, and without

Alternative	What it would have offered	Why it was not chosen
		a nightly extract-transform-load pipeline to build and maintain. A full data warehouse would be revisited only if reporting outgrows the replica.
Storing seat holds in the relational database	One store instead of two, and no dependency on Redis.	Holds are high-churn, short-lived rows. Persisting them adds write load and index churn to the busiest table for data that is worthless after five minutes, and expiry would require a sweeping background job instead of a TTL.

6.4.3 Tradeoffs Accepted

No design is free, and the chosen one carries three costs that are accepted knowingly.

- A single write primary is a scaling ceiling. All writes funnel through one instance. This is acceptable for twenty theaters, and the partition-by-theater scheme is precisely what would make a later shard-by-theater migration feasible if the chain expands substantially.
- Replica reads are eventually consistent. A replica may lag the primary by a fraction of a second, so seat-availability reads during checkout are directed to the primary while ordinary browsing reads are served from a replica. The rule is simple: anything a customer is about to act on financially is read from the primary.
- A second store adds an operational dependency. If Redis becomes unavailable, the system degrades rather than fails: the application falls back to holding seats through short database transactions, which is slower but correct.

6.5 Traceability to Requirements

Each data management decision traces to a requirement stated earlier in this SRS.

Requirement (Sections 2–3)	Decision	Where described
Website and kiosks must display the same live availability	One shared database reached only through the Data Access Layer	6.2, 6.3.2
A ticket must be uniquely identifiable and never sold twice	Unique constraints on barcode and on (showtime_id, seat_id)	6.3.1, 6.3.5
Tickets are held for five minutes during checkout, then released	Redis keys with a five-minute TTL	6.3.2, 6.3.5

Requirement (Sections 2–3)	Decision	Where described
The system supports at least 1,000 concurrent users	Read replicas, connection pooling, partitioning by theater	6.2.1, 6.3.4
Payment is processed by an external processor	Tokenized payment references; no card data stored	6.3.6
Staff may override an erroneous purchase	Immutable AUDIT_LOG rows for every override	6.3.3, 6.3.6
Management can report tickets sold and price per movie	Dedicated reporting replica queried with SQL	6.2.1, 6.4.2
Student, military, and senior discounts apply on weekdays	DISCOUNT_RULE table referenced by each ticket	6.3.3

6.6 GitHub Repository

The complete SRS document, including this section and all figures, is maintained in the project repository. All commits for this deliverable are authored by Benjamin Nguyen, who is the sole member of this project team.

Repository: <https://github.com/nguyenbenjamin2648-sketch/CS-250>

Deliverable 4 materials are stored under assignment4/ and include this document, the architecture diagram, and the logical data model diagram.